

get_installed_certificates

REFERENCE

API consumer

list the certificates installed in the reader's trust store

Overview

`get_installed_certificates` lists the certificates in the reader's trust store, returning per-certificate metadata — name, type, issuer/subject, serial, public-key algorithm, and validity dates — under `installedCerts.certInfo[]`. It is read-only and returns **no private-key material**. Use it to audit the certificate inventory and to confirm a certificate exists (and is valid) before referencing it from `config_endpoint` or `set_wifi`.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	v1.1 (API Version Reporting)
Related	install certificate , delete certificate , config_endpoint

When to Use

Use `get_installed_certificates` to audit or verify the reader's certificate inventory:

- **Before configuring TLS** — confirm the `ca_cert` / `client_cert` referenced by `config_endpoint` (`MQTT_TLS`) or `set_wifi` (Enterprise) is installed and of the right `type`.
- **Rotation planning** — check `validTill` (expiry) and `validFrom` (not-yet-valid) to schedule renewals before TLS handshakes start failing.
- **Trust audit** — review `issuerName` / `subjectName` / `serial` to confirm the expected CA chain.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request.

See [Topics and Channels](#).

Key Fields

- `certInfo[].type` (response) — the certificate's scope (`mqtt`, `wifi`, `filestore`, `client`, `server`); a certificate installed for one type is not usable by another.
- `certInfo[].name` (response) — the certificate's name. Published examples show component identifiers (`ca_cert`, `client_cert`), not the logical name given to `install_certificate`; which the reader reports is unresolved.
- `certInfo[].validTill` / `validFrom` (response) — the validity window; an expired or not-yet-valid certificate causes TLS handshake failures.
- `certInfo[].issuerName` / `subjectName` / `serial` (response) — identify the certificate and its CA chain (note `serial` is truncated — see [Response Semantics](#)).
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request is a minimal envelope — just `command` (the plural literal) and `requestId`. The command returns the full inventory; there are no filter parameters.

Request payload

```
1 {
2   "command" : "get_installed_certificates" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_installed_certificates	Fixed command name — plural get_installed_certificates (see the command-name below).
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response.

Request Semantics

- **Command literal.** Send `command: "get_installed_certificates"`. An unknown or bogus command value is silently dropped (no error, no reply), so a missing reply usually means a wrong command string.

Response Payload

On success the device returns the envelope (`command` , `requestId` , `apiVersion` , `response`) with the inventory under `installedCerts.certInfo[]` . The **Example** tab shows a five-certificate trust store: CA certificates for `filestore` , `wifi` and `mqtt` , and client certificates for `mqtt` and `wifi` .

Response payload

```
1  {
2    "command" : "get_installed_certificates" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.21" ,
5    "installedCerts" : {
6      "certInfo" : [
7        {
8          "name" : "ca_cert" ,
9          "type" : "filestore" ,
10         "issuerName" : "ExampleRoot" ,
11         "publicKeyAlg" : "RSA" ,
12         "publicKey" : "RSA" ,
13         "serial" : "ed:83:dd:4a:47:dd:07:98" ,
14         "subjectName" : "ExampleRoot" ,
15         "validFrom" : "2024-07-01T07:00:00Z" ,
16         "validTill" : "2034-02-28T15:18:47Z"
17       } ,
18       {
19         "name" : "ca_cert" ,
20         "type" : "wifi" ,
21         "issuerName" : "CA" ,
22         "publicKeyAlg" : "RSA" ,
23         "publicKey" : "RSA" ,
24         "serial" : "1e:e0:12:f6:75:68:39:f1" ,
25         "subjectName" : "CA" ,
26         "validFrom" : "2024-07-01T07:00:00Z" ,
27         "validTill" : "2025-04-16T07:26:35Z"
28       } ,
29       {
30         "name" : "ca_cert" ,
31         "type" : "mqtt" ,
32         "issuerName" : "Example-mqtt-ca" ,
33         "publicKeyAlg" : "RSA" ,
34         "publicKey" : "RSA" ,
35         "serial" : "7d:1b:c5:9f:ea:ff:0f:56" ,
36         "subjectName" : "Example-mqtt-ca" ,
37         "validFrom" : "2024-07-01T07:00:00Z" ,
38         "validTill" : "2038-01-19T03:14:07Z"
39       } ,
40       {
41         "name" : "client_cert" ,
42         "type" : "mqtt" ,
43         "issuerName" : "RootCA" ,
44         "publicKeyAlg" : "RSA" ,
45         "publicKey" : "RSA" ,
46         "serial" : "1" ,
47         "subjectName" : "Client" ,
48         "validFrom" : "2024-07-01T07:00:00Z" ,
49         "validTill" : "2030-05-06T08:06:57Z"
```

```
50     },
51     {
52         "name" : "client_cert" ,
53         "type" : "wifi" ,
54         "issuerName" : "CA" ,
55         "publicKeyAlg" : "RSA" ,
56         "publicKey" : "RSA" ,
57         "serial" : "26:00:16:e0:c4:a1:b5:00" ,
58         "subjectName" : "CLIENT" ,
59         "validFrom" : "2024-07-01T07:00:00Z" ,
60         "validTill" : "2025-04-16T07:27:52Z"
61     }
62 ]
63 },
64 "response" : {
65     "code" : 0 ,
66     "description" : "Success"
67 }
68 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_installed_certificates	Echoes the ex command (plu
2	requestId *	string	—	abc123	Echoes the re requestId — reply to requ
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.21	API version re this command per-handler, n wide.
4	installedCerts	object	—	—	The certificate Present on suc (Not in the env required set.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	certInfo	array	—	—	One entry per certificate. Each requires type, issuerName, subjectName, validFrom, validTill.
6	name	string	—	Reader Main Certificate	The name of the certificate. Even published examples carry a certificate component identifier here (ca_certificate, client_certificate) than the logical install_certificate accepts; which the reader reports unresolved.
7	type*	enum	client server mqtt wifi filestore	server	Certificate scope. Certificates are categorized by type — one for mqtt is not for wifi.
8	installTime	string	date-time	2022-07-01T03:01:31.464Z	Install timestamp. Optional — the reader often omits it.
9	issuerName	string	—	FX9600EE5729	Issuer common name

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
10	publicKeyAlg	enum	RSA RSAMD5 RSA_SHA_1 RSA_SHA_256 RSA_SHA_384 RSA_SHA_512	RSA	Public-key algorithm reported by the device (e.g. RSA).
11	publicKeyField	string	—	091234acgbe12	Public-key field device current populates this algorithm name than key bytes Response Semantics
12	serial	string	—	ed:83:dd:4a:47:dd:07:98	Certificate serial number in colon-hex — the first 6 bytes of the full certificate (the device true serial number see Response Semantics).
13	subjectName	string	—	FX9600EE572	Subject common name
14	validFrom	string	date-time	2024-07-01T07:00:00Z	Start of the valid time window.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
15	validity	string	date-time	2034-02-28T15:18:47Z	End of the validity window.
16	response	object	—	—	Status envelope read.
17	code	integer	0-101	0	0 = Success; non-zero = error. Requires contract. See Codes below.
18	description	string	—	Success	Human-readable

Response Semantics

- serial** is truncated to 8 bytes. The device returns the certificate serial as colon-delimited hex, but only the **first 8 bytes** of the (typically 20-byte) real serial — e.g. `7d:ef:d4:c3:2d:39:4a:0b`. Combine **serial** with **name** + **type** + **issuerName** to identify a certificate uniquely; **serial** alone is not a globally unique identifier.
- Private keys are never listed.** The inventory returns public certificate metadata only. A **client_key** installed via [install_certificate](#) does **not** appear in `certInfo[]` — only its paired **ca_cert** / **client_cert** do. There is no way (by design) to read back private-key material.
- publicKey** and **publicKeyAlg** quirks. The device currently populates **publicKey** with the algorithm name (e.g. `RSA`) rather than actual key bytes — treat **publicKey** as informational, not as usable key material. **publicKeyAlg** reports the certificate's public-key algorithm — an `RSA` variant (see the allowed values under [Response Payload](#)).

Response Codes

`get_installed_certificates` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the inventory was retrieved (an empty trust store still returns 0 with an empty/absent <code>certInfo</code>).
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the certificate list at this time. Retry after a short delay; if persistent, reboot.
9	File not found (<code>IOT_STATUS_FILE_NOT_FOUND</code>) — a backing certificate file could not be found.
21	Certificate not found (<code>IOT_ERROR_CERT_NOT_FOUND</code>) — the requested certificate is not installed.
101	Default error — generic catch-all.

[Codes](#) 3 / 9 / 21 are this command's mapped failure codes. The request takes no parameters; send exactly `command` + `requestId`.

Examples

Publish the plural `get_installed_certificates` with a `requestId`; the device replies with the inventory shown under [Response Payload](#). The command always returns the full list and takes no parameters (there are no request variants).

Notes and Caveats

Read-only, but still sensitive. The inventory exposes no secrets (no private keys), but a certificate list reveals the device's PKI/trust topology — redact it in shared logs, tickets, and screenshots. For the concept-level view, see [TLS and Authentication](#) and [Certificate Management](#).

Related:  [TLS and Authentication](#) ·  `install certificate` ·  `delete certificate` ·  `config_endpoint` ·  `set_wifi` ·  [Response Codes](#)